CHARUSAT®
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 7: Queue Implementation using Array and Linked List, with Structure Simulation

Practical Aim:

To implement queue using array and linked list, and apply queue-based logic to number generation, and structure simulation problems.

Problem 1 :

A government office has a token counter that issues tokens to visitors. The counter can hold at most n tokens at a time. New visitors join from one end and are served from the other end in the order they arrived. If the counter is full, no new token can be issued, and if it is empty, no one can be served. Given a sequence of join and serve operations, implement this fixed-capacity token system and print the current front token after each operation. Report an error if an operation cannot be performed.

Describe how your solution tracks the front and rear of the queue. What happens to your front and rear pointers over time as join and serve operations keep happening, and does your solution handle the case where space appears to run out even though slots have been freed?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-7

Output:

```
PS C:\Users\ADMIN\OneDrive\Desktop\DSA> cd "c:\Users\ADMIN\OneDrive\Desktop\DSA\Practical-7\"
5  is  current front token
40  is successful join
5  is  current front token
5  successful  served
10  is  current front token
10  is  current front token
10  successful  served
40  is  current front token
50  is successful join
40  is  current front token
PS C:\Users\ADMIN\OneDrive\Desktop\DSA\Practical-7>
```

Problem Faced:

The main challenge was deciding how to track the front and rear positions correctly as join and serve operations kept happening. Using a simple (non-circular) array, the rear index only moves forward on every join and never moves back, so after enough serve operations the rear can reach the end of the array and the queue gets reported as full even though several slots at the front have already been freed. It was also necessary to make sure overflow and underflow are reported clearly instead of writing outside the array or reading an invalid element.

Outcome/Conclusion:

A fixed-capacity token queue was implemented using an array of size n along with front and rear indices, both initialised to indicate an empty queue. A join

(enqueue) places the token at rear and increments rear, provided rear has not reached the last index; a serve (dequeue) removes the token at front and increments front, provided the queue is not empty. The current front token is printed after every successful operation, and an error is reported when join is attempted on a full queue or serve on an empty one.

Front and rear are simple indices into the array: rear increases by one on every join and front increases by one on every serve, and neither index ever moves backward. Because of this, the usable space before front is never reused, so even though those front slots have been freed by earlier serve operations, the queue can still report being full once rear reaches the end of the array. The implementation does not reclaim that space; a circular array would be needed for that.

Problem 2 :

A hospital emergency ward receives patients continuously and attends to them strictly in the order they arrived, with no upper limit on how many can be waiting at once. New patients are added at the back and the doctor always attends to the patient at the front. Given a sequence of arrive and attend operations, implement this unlimited patient queue and print the current front patient after each operation.

Describe how your solution differs from Problem 1 in how it manages capacity. What happens in your solution when the attend operation is called on an empty ward with no patients waiting?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-7

Output:

```
PS C:\Users\ADMIN\OneDrive\Desktop\DSA> cd "c:\Users\ADMIN\OneDrive\Desktop\DSA\Practical-7\"
7-2.cpp -o PRA7-2 } ; if ($?) { .\PRA7-2 }
5  New patients are added
5  is the current front patient
10  New patients are added
5  is the current front patient
40  New patients are added
5  is the current front patient
5  patient is arrive
10  is the current front patient
10  is the current front patient
10  patient is arrive
40  is the current front patient
50  New patients are added
40  is the current front patient
PS C:\Users\ADMIN\OneDrive\Desktop\DSA\Practical-7>
```

Problem Faced:

The main challenge was managing dynamic nodes correctly: a new node has to be created for every arrival and freed on every attend, and both the front and rear pointers must be kept consistent, including the case where the queue becomes empty and both pointers must be reset together. It was also necessary to handle the attend operation safely when no patient is waiting, without dereferencing a NULL pointer.

Outcome/Conclusion:

The unlimited patient queue was implemented using a singly linked list with separate front and rear pointers. An arrive operation creates a new node and

links it after the current rear (updating rear to the new node, and also updating front if the queue was empty); an attend operation removes the node at front, frees it, and moves front to the next node, resetting rear to NULL if the queue becomes empty. The current front patient is printed after every operation.

Unlike Problem 1, this queue has no fixed-size array and therefore no overflow condition: it grows one node at a time as patients arrive and shrinks as they are attended to, using only as much memory as is currently needed. When attend is called on an empty ward, the program checks front for NULL before removing anything and reports that no patient is waiting instead of accessing invalid memory.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation:

[Problem 1] A simple array queue eventually reports being full even when slots have been freed at the front. Explain why this happens structurally — not by tracing an example, but by describing what the front and rear pointers represent and what their movement pattern causes over time.

Sol: The front and rear pointers in a simple array queue represent the boundaries of the currently occupied region of the array, and both move only in one direction: front advances every time an element is removed, and rear advances every time an element is added. Neither pointer is ever reset backward to reuse a slot once it has been passed over. As a result, every served token permanently retires that array index for the rest of the program's run, even though the token stored there is gone. Over time the "used-up but empty" region before front keeps growing, and once rear reaches the last index of the array no further elements can be added, regardless of how many logically empty slots exist before front. This is a structural property of a simple (non-circular) array, not of any particular sequence of operations.

[Problem 2] Your linked list queue has no capacity limit. If arrive operations vastly outnumber attend operations over a long period, what real consequence would this have on the system, and where in your code is that consequence invisible?

Sol: If arrivals vastly outnumber attends, the number of nodes in the linked list keeps growing without bound, so the queue consumes more and more heap memory over time. Eventually this can exhaust the memory available to the program, causing a future node allocation to fail, slow the system down due to excessive memory usage, or cause the operating system to terminate the process. This consequence is invisible in the code because the arrive operation only ever allocates one new node and links it in — there is no size check, no counter being compared against a limit, and no point in the program where the growing memory usage is observed or reported, so the queue appears to accept arrivals indefinitely until the underlying system itself runs out of resources.